SPRINT 16 · DEMOSTRADOR · S16.1

Despliegue y documentación

Un modelo que solo vive en tu portátil no es un producto. Esta semana lo conviertes en un **demostrador** que otra persona pueda ejecutar con solo el README.

Qué se entrega y cuándo

Fecha	Qué
4/12	Demostrador funcionando (aunque sea básico) y esqueleto de la memoria
9/12	Sesgos mitigados, demostrador y **model card** casi listos; memoria con todos los apartados
10/12	**Checklist de congelación** completo y **último push antes de las 14:30**

Lunes 7 y martes 8/12: **sin clase**. Buen momento para avanzar la memoria en equipo. Lo que no esté subido a las 14:30 del 10/12, no cuenta.

El demostrador no reentrena

Carga un modelo **ya entrenado** y un archivo `metadatos.json` con lo que la interfaz necesita: qué campos pedir, rangos válidos, categorías, con qué datos y versiones se entrenó.

```
demostrador/
├── logica.py          # validar + predecir (sin Streamlit)
├── app.py             # interfaz Streamlit: capa fina
├── api.py             # (opcional) FastAPI: /salud y /predecir
├── tests/             # pruebas de sentido común
├── requirements.txt # versiones fijas
└── README_demostrador.md
```

La lógica, separada de la interfaz

- La lógica vive en `logica.py` **sin nada de Streamlit**: se puede probar y reutilizar en el cuaderno y en la API.
- Tres rasgos de una buena lógica:
  1. **Errores en español y accionables**: qué campo falla, qué se esperaba y qué se recibió.
  2. El resultado incluye el **aviso de incertidumbre** (métrica, límites, uso no permitido).
  3. **Devuelve datos, no imprime**.

```
def mensaje_errores(errores):                          # Ejercicio 2
    if not errores: return ""
    return "No se puede calcular todavía:\n" + "\n".join(f"- {e}" for e in errores)
```

Pruebas de sentido común

> Un modelo puede fallar **sin dar error**: devolver probabilidades absurdas.

```
def test_riesgo_alto_puntua_mas():
    alto = predecir({**reserva_ejemplo, "tarifa": "flexible", "deposito": "sin_deposito"})
    bajo = predecir({**reserva_ejemplo, "tarifa": "no_reembolsable", "deposito": "total"})
    assert alto["probabilidad"] > bajo["probabilidad"]
```

Mini-reto: **3 pruebas** propias y **vídeo de 90 s** como plan B (por si falla internet el día de la defensa).

Sesgos, model card y memoria

Sesgos

Tutoría de sesgos y visualización. **Mitiga** lo detectado y **vuelve a medir**; anota en el registro de experimentos.

Model card

`plantilla_model_card.md`: para qué sirve, con qué datos, cómo rinde, **qué no debe usarse**, riesgos.

Memoria técnica

10–15 páginas: resumen, problema, datos, metodología, modelado, evaluación, **IA responsable**, demostrador, gestión, conclusiones.

Mapa de requisitos **R1–R11**: comprueba que cada uno tiene su sección.

Lista de reproducibilidad y congelación

- [ ] `requirements.txt` con **versiones fijas** (`pip freeze` de lo que usas).
- [ ] El modelo se crea con un script o cuaderno (`entrenar`) y el README dice cómo.
- [ ] **Clona tu repo en un Colab nuevo** y comprueba que todo funciona.
- [ ] Demostrador, model card y memoria completos; evaluación e IA responsable revisadas.
- [ ] Vídeo de reserva subido; coevaluación entregada.
- 🔒 **14:30 del 10/12: congelación.** Último *push*.

S16 · DEMOSTRADOR · EN EL REPOSITORIO DEL CURSO

Ahora, a practicar

Lo que viene a continuación está en tu repositorio; no hace falta buscar ejercicios fuera.

CUADERNO
S16_01_demostrador — 45 min
Práctica del demostrador
sprints/S16/notebooks/S16_01_demostrador.ipynb

RETO
Reto del sprint — 40 min
Tutoría de sesgos y visualización (franja por equipo)
sprints/S16/guia/Reto_Sprint_16.md

RETO
Reto del sprint (sigue) — 65 min
Mitigar sesgos detectados y volver a medir
sprints/S16/guia/Reto_Sprint_16.md

RETO
Reto del sprint (sigue) — 35 min
Model card del modelo final
sprints/S16/guia/Reto_Sprint_16.md

RETO
Reto del sprint (sigue) — 35 min
Checklist de congelación y coevaluación
sprints/S16/guia/Reto_Sprint_16.md